

[Project Name]
Documentation

Alejandro Garcia Jr

[Member B]

[Member C]

[Member D]

[Member E]

9/27/2026

Contents

Change Log	2
1 Requirements Analysis	3
2 User Story	4
3 Key Technical Challenges	5
4 What's New?	6
5 Workflow	7
6 Use Cases	8
6.1 Alejandro Garcia Jr – Use Cases	8
6.1.1 UC: Search for Artist – Fan	8
6.1.2 UC: Review Report – Admin	9
6.1.3 UC: Create Event – Artist	11
6.2 <i>[Team Member Name]</i> – Use Cases	12
7 Use Case Diagrams	13
8 System Sequence Diagram	15
9 Domain Model	16
10 Operation Contracts	17
11 Test Cases	19
11.1 Test Cases for Iteration 1	19
11.2 Test Cases for Iteration 2	19
11.3 Test Cases for Iteration 3	19
11.4 Test Cases for Final Iteration	19

Change Log

Watch out: Treating early iterations as "draft"

Do not use placeholder text or "to be decided" sections anywhere in this document. Each iteration should be written as a complete, reviewable deliverable – assume it is final unless a later iteration explicitly revises it.

Iteration 1:

- *[What was created/changed in Iteration 1]*

Iteration 2:

- *[What was created/changed in Iteration 2]*

Iteration 3:

- *[What was created/changed in Iteration 3]*

Final Iteration:

- *[What was created/changed in the Final Iteration]*

1. Requirements Analysis

Watch out: Analysis before implementation

Finish this section and the Use Cases section *before* you discuss any classes, code, or UI screens elsewhere in the document. Every planned feature must trace to at least one requirement and one use case. Avoid phrases like "we plan to implement ..." without first explaining *why* the feature is needed from the user's perspective.

Customer / Stakeholder Perspective: *[Describe who the end users are and what they are trying to achieve, in their own words.]*

Functional Requirements:

- *[Functional requirement 1]*
- *[Functional requirement 2]*
- *[Functional requirement 3]*

Non-Functional Requirements / Environment:

- *[Platform, performance, deployment constraints, etc.]*

Requirements Clarification Questions

Watch out: Track open questions

List any clarification questions your team has raised with the client or instructor, and the answers once given. Update this list every iteration rather than deleting resolved items – it shows the grader your requirements evolved deliberately.

- **Q:** *[Question]*
 A: *[Answer, or "Pending"]*

2. User Story

Watch out: Requirements traceability

Every requirement above should be reflected in at least one user story, and every user story should later map to a use case. Keep IDs or consistent wording so a reader can trace a requirement all the way through to the System Sequence Diagrams.

- As a *[role]*, I want to *[goal]* so that *[benefit]*.
- As a *[role]*, I want to *[goal]* so that *[benefit]*.
- As a *[role]*, I want to *[goal]* so that *[benefit]*.

3. Key Technical Challenges

- *[Challenge 1 and how it will be approached]*
- *[Challenge 2 and how it will be approached]*

Key Risks

- *[Risk 1]*
- *[Risk 2]*

Competition

- *[Competitor / existing solution 1 – what it does, what it lacks]*
- *[Competitor / existing solution 2]*

4. What's New?

[Summarize what is novel about this project compared to existing solutions listed under Competition.]

5. Workflow

Watch out: System boundary

Make sure the workflow diagram clearly separates what the *actor* does from what the *system* does. Actors should never be shown performing system responsibilities (e.g., "system authenticates," not "user authenticates"), and external systems should not be drawn as if they were internal components.

[Insert workflow / activity diagram here, e.g.
`\includegraphics[width=\textwidth]{workflow.png}`*]*

6. Use Cases

Watch out: Use cases are not UI click-throughs

Do not write use cases as a sequence of UI actions ("user clicks button, system shows page"). Describe the *system's responsibilities and guarantees* – business rules, alternative flows, and constraints – not screen navigation. UI details will change over time; system behavior should be the stable part of the documentation.

Watch out: Fully-dressed format required

Each team member is responsible for exactly 3 use cases in *fully-dressed* form (Primary Actor, Preconditions, Postconditions, Main Success Scenario, Extensions). A one-line summary or a bare actor/system table is not sufficient for this deliverable.

Actors Overview

- **Fan** – discovers and follows artists, messages them, and books events.
- **Artist** – manages a public profile, proposes events, and communicates with fans.
- **Admin** – moderates content and accounts by reviewing user-filed reports.
- *[System actor (e.g., Payment Processor, Push Notification Service) – add if/when identified]*

6.1 Alejandro Garcia Jr – Use Cases

6.1.1 UC: Search for Artist – Fan

Primary Actor

Fan

Scope ArtistConnect chat application

Level User goal

Stakeholders & Interests

- **Fan:** wants to quickly find artists matching their interests.
- **Artist:** wants to be discoverable by relevant fans.

Preconditions • The Fan is authenticated and logged into the system.

Postconditions (Success Guarantee) • A list of artists matching the Fan's search criteria was displayed.

- If the Fan selected a result, the corresponding action (profile view, chat, follow, or booking request) was recorded by the system.

Main Success Scenario

1. The Fan selects the search feature.
2. The system prompts the Fan for search criteria (name, genre/style, location).
3. The Fan enters one or more criteria and submits the search.
4. The system queries registered artists matching the given criteria.
5. The system displays a list of matching artists, each showing name, genre, location, and profile thumbnail.
6. The Fan selects an artist from the results list.
7. The system displays the selected artist's full profile.
8. The Fan chooses a follow-up action: start a direct chat, follow the artist, or book the artist.
9. The system performs the selected action and confirms it to the Fan.

Extensions

- 2a. The Fan submits the search with no criteria entered: the system prompts the Fan to enter at least one search criterion.
- 4a. No artists match the given criteria: the system displays a "no results found" message and suggests broadening the search.
- 6a. The Fan exits without selecting a result: the use case ends with no further action taken.
- 8a. The Fan chooses "Start Chat" and a conversation already exists with that artist: the system opens the existing conversation instead of creating a new one.
- 8b. The Fan chooses "Book": the system starts the booking-request flow (a separate use case) and, on submission, notifies the Artist that a booking request is pending.

Special Requirements

- Search results should be returned in under 2 seconds for typical query loads.
- Result lists longer than 20 artists are paginated.

Frequency of Occurrence

High – expected to be one of the most common Fan actions.

6.1.2 UC: Review Report – Admin**Primary Actor**

Admin

Scope

ArtistConnect chat application

Level

User goal

Stakeholders & Interests

- **Admin:** needs enough context to make a fair, auditable moderation decision.
- **Reporting user (Fan or Artist):** wants inappropriate content or behavior addressed.

- **Reported user (Fan or Artist):** has an interest in fair treatment and being informed of any action taken.

Preconditions • The Admin is authenticated with administrative privileges.

- At least one report exists with status "Pending," filed by a Fan or an Artist against a message, profile, or event.

Postconditions (Success Guarantee) • The report's status was updated to "Resolved," with the decision and acting Admin recorded.

- Any moderation action taken (dismiss, warn, delete content, suspend/ban account) was applied to the reported content or account.
- Both the reporting user and the reported user were notified of the outcome.

Main Success Scenario 1. The Admin opens the reports queue.

2. The system displays pending reports, each showing the reporter, reported content or user, reason, and timestamp.
3. The Admin selects a report to review.
4. The system displays full report details, including the reported message, profile, or event, and the reporter's stated reason.
5. The Admin evaluates the report and selects an action: dismiss, warn the user, delete the content, or suspend/ban the account.
6. The system applies the selected action.
7. The system records the resolution (Admin identity, action taken, timestamp) and sets the report status to "Resolved."
8. The system notifies the reporting user that their report was reviewed, and notifies the reported user of any action taken against them.

Extensions 3a. The selected report was already resolved by another Admin: the system informs the Admin and returns to the queue.

- 5a. The Admin dismisses the report as unfounded: the system records the dismissal and notifies only the reporting user.
- 5b. The Admin suspends or bans the reported user's account: the system also terminates any active sessions for that account.
- 6a. The reported content was already removed (e.g., deleted by its owner): the system notes this and proceeds directly to logging the resolution.

Special Requirements • The reports queue must support filtering by report type (message, profile, or event).

- All resolutions must be permanently logged for audit purposes.

Frequency of Occurrence

Low to moderate – driven by user-filed reports rather than a routine action.

6.1.3 UC: Create Event – Artist

Primary Actor

Artist

Scope ArtistConnect chat application

Level User goal

Stakeholders & Interests • **Artist:** wants to promote and sell tickets to an upcoming event.

- **Admin:** needs to approve proposed pricing before the event is publicly visible.
- **Fan:** wants accurate, trustworthy event listings.

Preconditions • The Artist is authenticated and has an account in good standing.

Postconditions (Success Guarantee) • A new event was created with status "Pending Approval" and routed to the Admin queue.

- Once approved, the event's status was updated to "Published" and it became visible/searchable to Fans, who were notified if they follow the Artist.

Main Success Scenario 1. The Artist selects "Create Event."

2. The system displays the event creation form.
3. The Artist enters the event details: title, date/time, location (physical or virtual link), description, capacity, and one or more proposed ticket prices/tiers.
4. The Artist submits the event.
5. The system validates the submitted data.
6. The system creates the event with status "Pending Approval" and adds it to the Admin approval queue.
7. The system confirms to the Artist that the event was submitted for review.
8. The Admin reviews and approves the proposed pricing (see UC: Review Event Pricing).
9. The system updates the event's status to "Published," notifies the Artist of the approval, and makes the event visible/searchable to Fans, notifying the Artist's followers.

Extensions 3a. The Artist saves the event as a draft instead of submitting: the system stores it with status "Draft" and does not route it for approval.

- 5a. The submitted data is missing required fields or contains an invalid date/price: the system flags the offending fields and asks the Artist to correct them before resubmitting.
- 8a. The Admin rejects the proposed pricing: the system notifies the Artist with the rejection reason(s); the Artist may revise and resubmit, returning to step 3.
- 9a. The Artist edits event details after publication: the system flags the event for Admin re-approval if the change affects pricing, otherwise the update is applied immediately.

Special Requirements • Ticket pricing must support multiple tiers (e.g., general admission, VIP).

- Artists should be notified promptly (e.g., push/email) of an approval or rejection decision.

Frequency of Occurrence

Moderate – expected whenever an Artist plans a new event.

Repeat the section above for each additional team member, with their own 3 use cases in the same fully-dressed format.

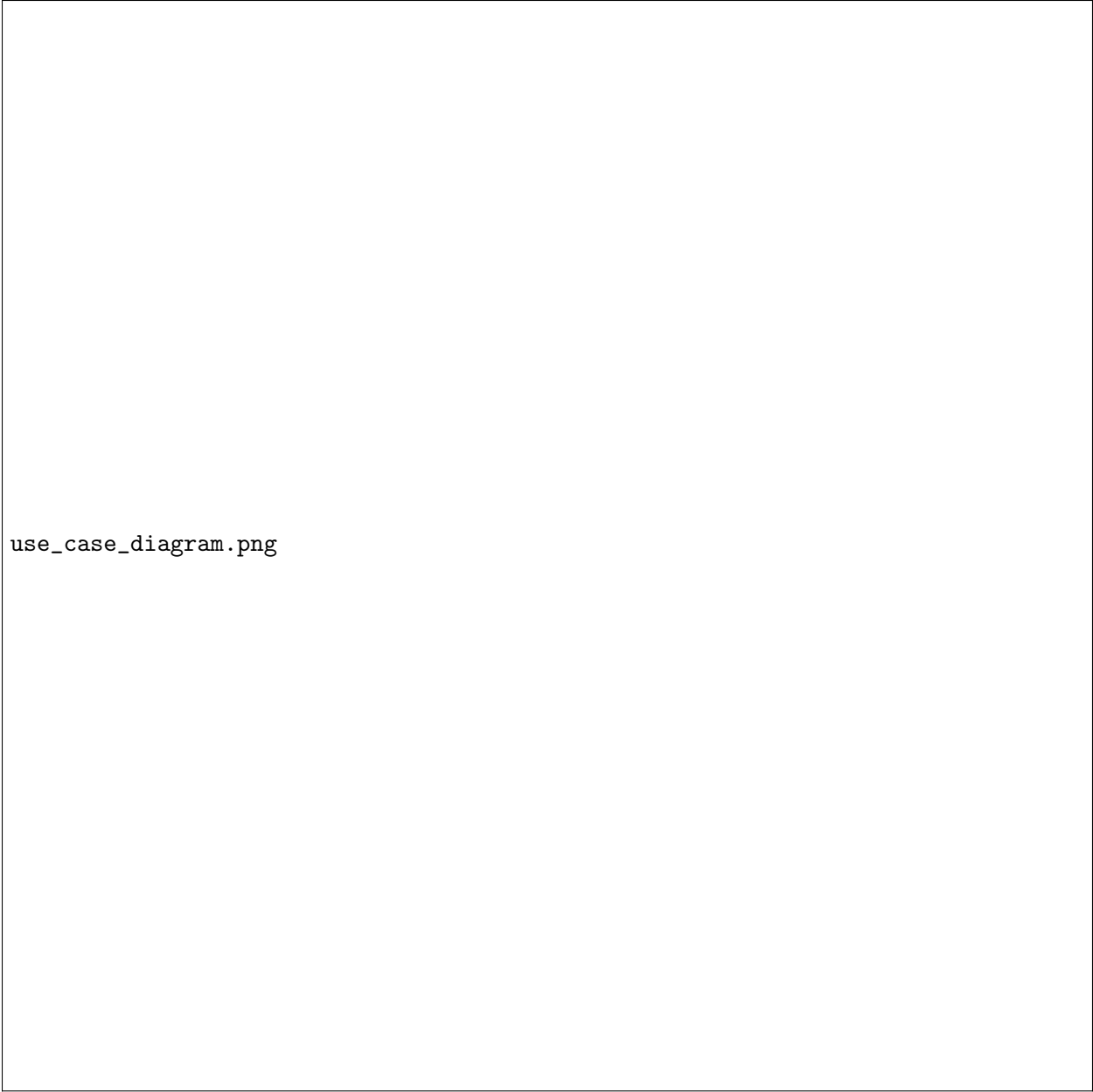
6.2 [Team Member Name] – Use Cases

[Add this member's 3 fully-dressed use cases here, following the format above.]

7. Use Case Diagrams

Watch out: Every use case must appear here

Every fully-dressed use case in the previous chapter – from every team member – should appear as an oval in this diagram, associated with its actor(s). If a use case is missing here, or a use case appears here with no matching fully-dressed writeup, that is a traceability gap graders will look for.



`use_case_diagram.png`

[This diagram currently shows only Alejandro's 3 use cases plus placeholder ovals for teammates. Regenerate `use_case_diagram.puml` (PlantUML source included alongside this document) once every member's use cases are finalized, replacing each dashed placeholder oval with the real use case name and its correct actor associations.]

8. System Sequence Diagram

Watch out: Consistency with use cases

Every scenario shown here should correspond to a use case described earlier, using the same name. Missing or renamed scenarios break traceability between Use Cases and SSDs.

[Scenario Name]

[Insert system sequence diagram here.]

Actor	System
1. <i>[step]</i>	
	2. * <i>[system response]</i>
3. <i>[step]</i>	

Repeat for each scenario.

9. Domain Model

Watch out: Domain model is not a class diagram

Use only nouns from the problem statement as domain classes. Do not include methods, data types, UI classes, or persistence/database details – those belong to design, not analysis. Show entities and the associations/multiplicities between them, not implementation.

[Insert domain model diagram here, e.g.
`\includegraphics[width=\textwidth]{domain_model.png}`*]*

10. Operation Contracts

Watch out: One contract per key system operation

An operation contract documents *what* a system operation guarantees, not *how* it is implemented. Postconditions describe instances created/deleted, associations formed/broken, and attributes changed – written in the past tense, as if describing what has already happened. Write one contract per team member’s use cases (typically the one system operation in the main success scenario that changes state).

Operation: searchArtists(criteria: SearchCriteria): List<ArtistSummary>

Cross Reference

UC: Search for Artist – Fan

Preconditions • The Fan is logged into the system.

Postconditions • A collection of Artist instances matching the given criteria (name, genre, location) was retrieved.
• No system state was created, modified, or deleted (read-only operation).

Operation: resolveReport(reportId: ReportID, action: ActionType): void

Cross Reference

UC: Review Report – Admin

Preconditions • The Admin is logged in with administrative privileges.
• A Report instance identified by **reportId** exists with status = "Pending."

Postconditions • The Report instance’s **status** attribute was changed to "Resolved."
• A new Resolution instance was created and associated with the Report, recording the acting Admin, the action taken, and a timestamp.
• If **action** = DeleteContent, the associated Message, Profile, or Event instance’s **status** attribute was changed to "Removed."
• If **action** = Suspend or Ban, the reported User instance’s **status** attribute was changed to "Suspended" or "Banned" respectively.
• A new Notification instance was created and associated with the reporting User.

- A new Notification instance was created and associated with the reported User, if `action` $\neq$ Dismiss.

Operation: `submitEvent(eventData: EventForm): Event`

Cross Reference

UC: Create Event – Artist

Preconditions • The Artist is logged in and owns an account in good standing.

Postconditions • A new Event instance was created with `title`, `dateTime`, `location`, `description`, `capacity`, and `proposedPrices` set from `eventData`.
• The new Event instance's `status` attribute was set to "Pending Approval."
• An association was formed between the new Event instance and the Artist who created it.
• The new Event instance was added to the Admin approval queue.

Repeat the operation contract pattern above for each additional team member's key use-case operation.

11. Test Cases

Watch out: Traceability of tests

Each test case should reference the requirement or use case it verifies. Keep test case IDs consistent across iterations so gaps in coverage are easy to spot.

11.1 Test Cases for Iteration 1

Target Users: *[Who]*

What is covered in the tests:

1. *[Area of functionality covered]*

Test Case: *[ID, e.g., T1]*

- **Description:** *[What this test checks]*
- **Action:** *[Function/action exercised]*
- **Related Requirements:** *[Preconditions]*
- **Expected Result:** *[Expected behavior]*
- **Test Result:** *[Actual behavior observed]*
- **Status:** *[Pass / Fail]*

Repeat the Test Case block above for each test in this iteration.

11.2 Test Cases for Iteration 2

[Same structure as Iteration 1.]

11.3 Test Cases for Iteration 3

[Same structure as Iteration 1.]

11.4 Test Cases for Final Iteration

[Same structure as Iteration 1.]